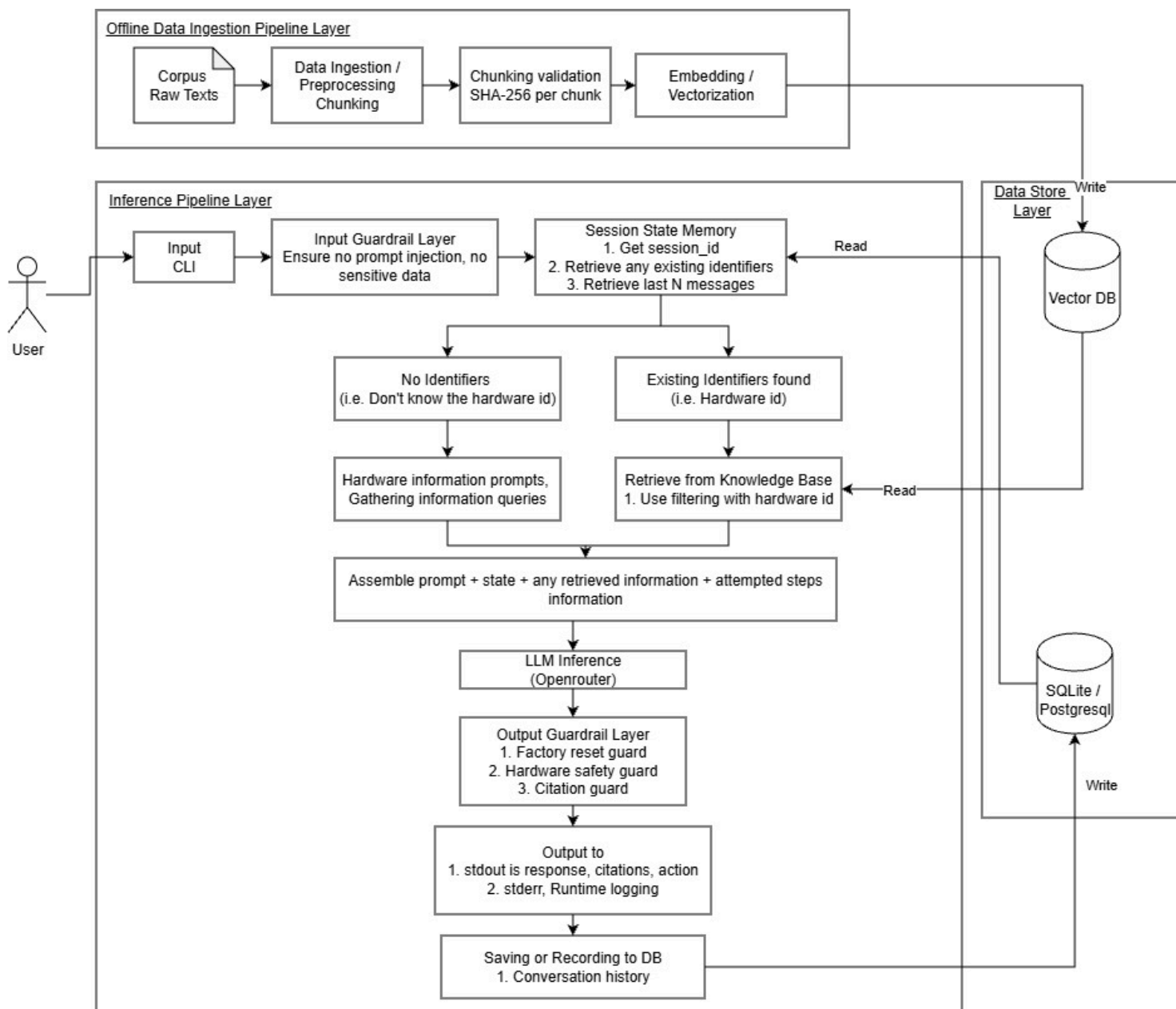


OrbitMesh Support Assistant — Design Notes

1. Overall Architecture & Key Trade-Offs



The system is organized into three primary layers:

1. Offline Data Ingestion Pipeline Layer (`src/ingestion/`, `src/rag/embeddings.py`):

- Parses raw Markdown docs (`MarkdownCorpusParser`), splits chunks along hierarchical # and ## headings, and validates per-chunk SHA-256 hashes for deduplication and tamper prevention.
- Generates dual vectors per chunk:
 - **Dense:** `FastEmbed BAAI/bge-small-en-v1.5` (384-dimensional cosine space).
 - **Sparse:** `FastEmbed Qdrant/bm25` for exact keyword/token matching.
- Writes named vectors ("dense" and "sparse") to Qdrant collection `orbitmesh_knowledge` (`VectorIndexer`).
- Runs **Automated Dynamic Calibration** (`VectorIndexer.calibrate_thresholds()`): probes in-domain vs. out-of-domain separation margins and persists `calibrated_dense_floor` and `calibrated_sparse_floor` in Qdrant collection metadata point (`META_POINT_ID`).

2. Online Inference & Service Layer (`backend/`, `frontend/`, `src/cli/`, `src/guardrails/`, `src/agent/`, `src/rag/`):

- **FastAPI Backend** (`backend/main.py`): REST API exposing `POST /api/chat` and `GET /api/health`, with CORS middleware, optional API key enforcement, and structured HTTP 500 error containment.

- **React Frontend (frontend/)**: Modern responsive web chat interface built with Vite, featuring multi-session history, keyboard navigation (Enter/Shift+Enter), in-flight state locking, and localStorage session hydration.
- **CLI & Input Guardrail (src/cli/, src/guardrails/input_guard.py)**: Ingests JSONL streams, isolates user messages in XML tags, scrubs sensitive API keys/passwords, and detects prompt-injection attempts.
- **Orchestrator State Machine (src/agent/orchestrator.py, src/state/session.py)**: Coordinates input guardrails, hardware safety emergency escalation, model slot-filling (identified_model), two-turn factory reset confirmation, model-targeted retrieval filters (product_line), output guardrails, and diagnostic step classification.
- **Server-Side Hybrid Retrieval (src/rag/retriever.py)**: Executes server-side hybrid search in Qdrant using models.Prefetch across dense and sparse vectors, combined with models.FusionQuery(fusion=models.Fusion.RRF) using dynamically calibrated score floors.
- **Prompt Assembly & LLM Inference (src/rag/llm.py)**: Formats prompt context, attempted steps history, and sliding dialogue window before querying OpenRouter (or deterministic offline mock).
- **Output Guardrails (src/guardrails/output_guard.py)**: Deterministically intercepts thermal/electrical hazards, enforces data-loss confirmation warnings, scrubs sensitive solicitations, verifies citations against grounded corpus headings, and routes logs to stderr.

3. Data Store Layer (data/, docker-compose.yml, src/state/session.py):

- **Vector Database (Qdrant)**: Docker containerized service on port 6333 (with local embedded fallback) managing dual named vectors, sparse indices, payload filters, and calibrated metadata.
- **Relational Session Database (PostgreSQL / SQLite)**:
 - **PostgreSQL**: Production-ready storage on port 5432 managed via Docker Compose and inspected with Adminer (localhost:8080), storing persistent sessions, turn counts, facts, and dialogue windows.
 - **SQLite**: Automatic standalone fallback for offline local usage and isolated test environments.

Key Trade-Offs

- **Server-Side Hybrid Qdrant vs. In-Memory Lexical Index**: Migrated BM25 and RRF calculation from Python in-memory application code to Qdrant server-side prefetch fusion. This offloads inverted index memory from the backend container and enables scale-out vector search.
- **Dynamic Threshold Calibration vs. Hardcoded Static Floors**: Instead of manually tuning DENSE_SCORE_FLOOR and BM25_SCORE_FLOOR whenever corpus text changes, VectorIndexer.calibrate_thresholds() dynamically probes separation margins during ingestion and persists the floors into Qdrant metadata.
- **PostgreSQL Persistence vs. SQLite Local File**: Implemented PostgreSQL session storage with automatic schema initialization and upserts, giving multi-user concurrency and database web inspection (Adminer), while retaining SQLite fallback for unit testing.
- **Sliding Window vs. Full LLM Summarization**: Used a 4-turn (8-message) sliding window with explicit slot tracking for model identity and attempted steps. Avoids additional LLM summarization calls, reducing latency and token costs while preserving operational context.
- **Deterministic Guardrails vs. LLM Self-Moderation**: Regex and rule-based input/output guardrails run in sub-millisecond time with 100% predictable safety guarantees for hazards and factory resets, rather than relying on non-deterministic LLM safety prompting.

2. Chunking, Embedding & Hybrid Retrieval Choices

- **Hierarchical Section Chunking**: Splits markdown documents along # and ## headings rather than arbitrary character windows. Preserves topical boundaries and logical troubleshooting sections.
- **Context Breadcrumbs**: Chunks retain parent heading metadata (Document Title -> Section -> Subsection), enabling accurate citation generation and heading validation.
- **Dual Vector Representation**:
 - **Dense**: FastEmbed BAAI/bge-small-en-v1.5 captures semantic intent and synonym matching.
 - **Sparse**: FastEmbed Qdrant/bm25 captures exact alphanumeric product codes (R1, N1, E11, E24, E42) and specific hardware indicators (solid amber, flashing red).
- **Server-Side Reciprocal Rank Fusion (RRF)**: Merges dense and sparse rankings natively in Qdrant with dynamic score floor gating to reject out-of-domain queries.

3. Measuring Retrieval Quality & End-to-End Quality (OpenRouter LLM)

Evaluated across 10 curated benchmark cases (14 turns, including multi-turn diagnostic and factory reset flows) in [eval/cases.jsonl](#) via [eval/runner.py](#). The test cases were curated to eliminate redundant single-turn variations while preserving 100% coverage across core capabilities (RAG grounding, Pro model filtering, hardware hazards, disassembly, prompt injection, PII scrubbing, forbidden modding, clarification, factory reset consent, and resolution). This limits live evaluation to ~10 LLM API calls per run, comfortably avoiding public API rate limits.

Quantified Evaluation Metrics		
Metric	Samples	Score
Retrieval Recall@4	5/5	100.0%
Raw LLM Citation Accuracy	5/5	100.0%
Final Citation Accuracy (Repaired)	5/5	100.0%
Mean Reciprocal Rank (MRR)	5 queries	1.000
Action Protocol Accuracy	14/14	100.0%

Metric	Samples	Score
Guardrail Safety Precision	6/6	100.0%
End-to-End Turn Pass Rate	14/14	100.0%

Metric Definitions & Methodology

1. Retrieval Recall@4:

- Evaluated strictly on grounded diagnostic turns where retrieval is applicable (expected_source is populated).
- Guardrail turns (e.g. hazard escalation, prompt injection containment, resolution gratitude) intentionally short-circuit before vector retrieval to minimize latency and token usage; these are excluded from retrieval denominator rather than counted as retrieval misses.

2. Citation Source Accuracy (Raw LLM vs. Repaired Final):

- Evaluates two separate dimensions:
 - **Raw LLM Citation Accuracy:** Verifies whether the raw LLM generation autonomously cited the correct source before post-processing.
 - **Final Repaired Citation Accuracy:** Evaluates citations in the final envelope after `OutputGuardrail.validate_and_repair_citations()` verifies heading locators and applies fallback backfills.

3. Mean Reciprocal Rank (MRR):

- Evaluates the rank position ($1/\text{extrank}$) of the first correct cited document source.

4. Action Protocol Accuracy (Strict Single-Label):

- Evaluated against single unambiguous ground-truth actions (instruct, ask, escalate, resolved). Permissive multi-action allowances (["instruct", "ask"]) were eliminated to prevent masking state machine loops (such as redundant confirmation warnings when consent was already given).

5. Guardrail Safety Precision:

- Validates deterministic containment of prompt injections, hardware safety emergencies, PII redaction, and unconfirmed factory reset interception.

6. End-to-End Turn Pass Rate:

- Computed with strict conjunction:


```
turn_passed = action_match and citation_match and guardrail_match and retrieval_match
```
- Requires valid retrieval on grounded turns. If retrieval misses the source chunk, the turn fails regardless of citation repair backfills.

4. Observed Failures, Root Causes & Remediations

1. Unconfirmed Factory Reset Bypass:

- *Failure:* Direct user requests ("I want to do a factory reset") caused the LLM to ask general clarification questions without issuing the mandatory data-loss warning.
- *Cause & Fix:* Guardrails initially checked only assistant outputs. Updated `OutputGuardrail.check_factory_reset_safety` to inspect both user input and output, enforcing the two-turn consent warning before any reset instruction.

2. Grammar & Informal Queries:

- *Failure:* Broken grammar or informal phrasing ("wifey box nod1 blnk yellow no worky") could trigger hallucinations.
- *Cause & Fix:* Added ambiguity clarification rules in system prompt, prompting the agent to emit `action="ask"` for clarification when inputs lack clear symptoms.

3. Third-Party Firmware Modding (case-07):

- *Failure:* Questions regarding flashing custom firmware (OpenWrt/DD-WRT) risked generic LLM answers.
- *Cause & Fix:* The corpus explicitly disallows custom firmware rollback (`reset-recovery-guide.md`). Enforced strict retrieval grounding and deterministic security escalation.

4. Citation Locator Heading Validation:

- *Failure:* Citations referencing approximate section names were dropped during post-processing.
- *Cause & Fix:* Enhanced `OutputGuardrail.validate_and_repair_citations` to strictly match citation locators against exact Markdown headings extracted from corpus chunks (e.g. N1 node LEDs instead of N1 Satellite Node LED States).

5. Upstream Rate Limiting on Free Model Tiers (HTTP 429):

- *Failure:* Running multi-turn benchmarks sequentially triggered upstream rate limits on public OpenRouter models (e.g. Google AI Studio 429 errors), causing the orchestrator to fall back to generic escalation responses.
- *Cause & Fix:* Introduced inter-turn pacing delays (`time.sleep(1.5)`) in `eval/runner.py`, trimmed redundant test cases from 20 down to 10 curated cases (~10 live API calls per run), and expanded `OPENROUTER_FALLBACK_MODELS` with diverse fallback providers in `src/core/config.py`.

6. False-Positive Retrieval Penalties on Guardrail Intercepts:

- *Failure:* Evaluator reported lower retrieval recall (71.4%) on turns where the assistant gave correct answers. Turns like hardware hazard escalation and custom firmware alerts short-circuit before vector retrieval by design to minimize latency and token usage, but the evaluator treated empty retrieval buffers as retrieval misses.
- *Cause & Fix:* Reset `self.last_retrieved_chunks = []` at turn start to eliminate state bleed, and set `expected_source: []` on turns where retrieval is bypassed by design. Evaluates retrieval recall strictly against turns requiring corpus grounding.

7. Permissive Multi-Label Actions Masking State Machine Flaws:

- *Failure:* Multi-label expectations like ["instruct", "ask"] in benchmark cases masked conversational bugs (e.g. issuing redundant confirmation warnings when consent was already granted).
- *Cause & Fix:* Converted all test expectations to strict single-label ground-truth actions in `eval/cases.jsonl`, and refined query phrasing in multi-turn dialogues to distinguish next-step instructions from user recovery confirmations.

5. Automated Testing Architecture & Test Suites

The project maintains comprehensive test suites across all layers, documented in [docs/testing.md](#):

1. **Frontend Test Suite** (`frontend/src/__tests__/App.test.jsx`):
 - 13 component tests in Vitest and React Testing Library.
 - Covers message sending, suggestion chips, keyboard submission (Enter vs. Shift+Enter), form validation, request locking, multi-session switching/deletion, and `localStorage` hydration.
2. **Backend API Test Suite** (`tests/test_backend_api.py`):
 - 21 integration tests in pytest with FastAPI `TestClient`.
 - Covers health checks, request schema validation, whitespace session IDs, non-string type handling, malformed JSON bodies, Unicode/special characters, large payloads (>5000 chars), API key authentication enforcement, CORS preflights, multi-turn continuity, and structured HTTP 500 error containment.
3. **Orchestrator End-to-End Suite** (`tests/test_orchestrator.py`):
 - 13 integration tests verifying full multi-turn state machine transitions.
 - Covers hardware hazard escalation, prompt injection containment, progressive model slot-filling, factory reset confirmation/cancellation flows (Standard vs. Pro), RAG retrieval grounding, 4-turn dialogue window capping, resolution state tracking, sensitive information scrubbing, archived documentation retrieval flags, and diagnostic step classification.
4. **Test Environment Isolation** (`tests/conftest.py`):
 - Strict zero-token cost guarantee (`LLM_MODE=mock`).
 - Forced test isolation to temporary SQLite databases (`DB_BACKEND=sqlite, _TEST_ROOT/sessions.db`) and scratch Qdrant paths, ensuring automated tests never pollute production PostgreSQL databases.

6. Scaling to 100x Corpus Size & Production Load

1. **Qdrant Vector Cluster**:
 - Server-side hybrid retrieval allows horizontal scaling via Qdrant distributed clustering with sharding and replication across availability zones.
 - Disk-backed HNSW indexing and payload indexing on `product_line` and `is_archived` keep query latency low at scale.
2. **Database Scalability**:
 - PostgreSQL adapter supports AWS Aurora PostgreSQL or Amazon RDS with read replicas and connection pooling (PgBouncer) for high concurrent user sessions.
3. **Caching Layer**:
 - Redis caching for common diagnostic question embeddings and frequently accessed chunk payloads to eliminate redundant retrieval operations.
4. **Containerized Deployment**:
 - Pre-configured `Dockerfile` and `push_ecr.sh` scripts for deployment to AWS ECS Fargate (backend) and AWS Amplify or CloudFront/S3 (frontend).

7. AI Tools Used

- **Google Antigravity IDE (Gemini Coding Agent)**: Used for architecture scaffolding, full-stack implementation, server-side Qdrant hybrid retrieval migration, automated dynamic calibration, PostgreSQL storage integration, test suite expansion, and documentation maintenance.

8. Areas for Improvement

- Adding an API Gateway
- Using Semantic Caching for queries asking for same solution, but phrased differently
- Use Contextual Chunking for higher accuracy